

```
pip install langchain_openai
```

```
WARNING: Ignoring invalid distribution ~vidia-cublas-cu12 (/usr/local/lib/python3.11/dist-packages)
WARNING: Ignoring invalid distribution ~vidia-cublas-cu12 (/usr/local/lib/python3.11/dist-packages)
Requirement already satisfied: langchain_openai in /usr/local/lib/python3.11/dist-packages (0.3.14)
Requirement already satisfied: langchain-core<1.0.0,>=0.3.53 in /usr/local/lib/python3.11/dist-packages (from langchain_openai) (0.3.55)
Requirement already satisfied: openai<2.0.0,>=1.68.2 in /usr/local/lib/python3.11/dist-packages (from langchain_openai) (1.75.0)
Requirement already satisfied: tiktoken<1,>=0.7 in /usr/local/lib/python3.11/dist-packages (from langchain_openai) (0.9.0)
Requirement already satisfied: langsmith<0.4,>=0.1.125 in /usr/local/lib/python3.11/dist-packages (from langchain-core<1.0.0,>=0.3.53->langchain-core) (0.1.125)
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in /usr/local/lib/python3.11/dist-packages (from langchain-core<1.0.0,>=0.3.53->langchain-core) (8.5.0)
Requirement already satisfied: jsonpatch<2.0,>=1.33 in /usr/local/lib/python3.11/dist-packages (from langchain-core<1.0.0,>=0.3.53->langchain-core) (1.33)
Requirement already satisfied: PyYAML>=5.3 in /usr/local/lib/python3.11/dist-packages (from langchain-core<1.0.0,>=0.3.53->langchain-core) (6.0.2)
Requirement already satisfied: packaging<25,>=23.2 in /usr/local/lib/python3.11/dist-packages (from langchain-core<1.0.0,>=0.3.53->langchain-core) (25.0)
Requirement already satisfied: typing-extensions>=4.7 in /usr/local/lib/python3.11/dist-packages (from langchain-core<1.0.0,>=0.3.53->langchain-core) (4.12.2)
Requirement already satisfied: pydantic<3.0.0,>=2.5.2 in /usr/local/lib/python3.11/dist-packages (from langchain-core<1.0.0,>=0.3.53->langchain-core) (2.10.6)
Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.11/dist-packages (from openai<2.0.0,>=1.68.2->langchain_openai) (4.6.2)
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.11/dist-packages (from openai<2.0.0,>=1.68.2->langchain_openai) (1.7.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from openai<2.0.0,>=1.68.2->langchain_openai) (0.27.0)
Requirement already satisfied: jiter<1,>=0.4.0 in /usr/local/lib/python3.11/dist-packages (from openai<2.0.0,>=1.68.2->langchain_openai) (0.8.2)
Requirement already satisfied: sniffio in /usr/local/lib/python3.11/dist-packages (from openai<2.0.0,>=1.68.2->langchain_openai) (1.3.1)
Requirement already satisfied: tqdm>4 in /usr/local/lib/python3.11/dist-packages (from openai<2.0.0,>=1.68.2->langchain_openai) (4.66.5)
Requirement already satisfied: regex>=2022.1.18 in /usr/local/lib/python3.11/dist-packages (from tiktoken<1,>=0.7->langchain_openai) (2024.11.6)
Requirement already satisfied: requests>=2.26.0 in /usr/local/lib/python3.11/dist-packages (from tiktoken<1,>=0.7->langchain_openai) (2.32.3)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.11/dist-packages (from anyio<5,>=3.5.0->openai<2.0.0,>=1.68.2->langchain_openai) (3.10.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.11/dist-packages (from httpx<1,>=0.23.0->openai<2.0.0,>=1.68.2->langchain_openai) (2025.1.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.11/dist-packages (from httpx<1,>=0.23.0->openai<2.0.0,>=1.68.2->langchain_openai) (1.0.7)
Requirement already satisfied: h11<0.15,>=0.13 in /usr/local/lib/python3.11/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->openai<2.0.0,>=1.68.2->langchain_openai) (0.14.0)
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.11/dist-packages (from jsonpatch<2.0,>=1.33->langchain-core<1.0.0,>=0.3.53->langchain-core) (3.0.0)
Requirement already satisfied: orjson<4.0.0,>=3.9.14 in /usr/local/lib/python3.11/dist-packages (from langsmith<0.4,>=0.1.125->langchain-core) (3.10.1)
Requirement already satisfied: requests-toolbelt<2.0.0,>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from langsmith<0.4,>=0.1.125->langchain-core) (1.0.0)
Requirement already satisfied: zstandard<0.24.0,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from langsmith<0.4,>=0.1.125->langchain-core) (0.24.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.11/dist-packages (from pydantic<3.0.0,>=2.5.2->langchain-core) (0.7.0)
Requirement already satisfied: pydantic-core==2.33.1 in /usr/local/lib/python3.11/dist-packages (from pydantic<3.0.0,>=2.5.2->langchain-core) (2.33.1)
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.11/dist-packages (from pydantic<3.0.0,>=2.5.2->langchain-core) (0.4.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests>=2.26.0->tiktoken<1,>=0.7->langchain_openai) (3.4.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests>=2.26.0->tiktoken<1,>=0.7->langchain_openai) (2.3.0)
WARNING: Ignoring invalid distribution ~vidia-cublas-cu12 (/usr/local/lib/python3.11/dist-packages)
```

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model_name="gpt-4o", api_key = "sk-proj-yl8CMOMeByMtdMvQuKC_p50upISvMrwBaPuAZAPhIXtBfNq2AF6N_ppZwWFPvwm4XK4CVPg0-tT3B1bkfJg")
```

```
pip install langgraph
```

```
WARNING: Ignoring invalid distribution ~vidia-cublas-cu12 (/usr/local/lib/python3.11/dist-packages)
WARNING: Ignoring invalid distribution ~vidia-cublas-cu12 (/usr/local/lib/python3.11/dist-packages)
Requirement already satisfied: langgraph in /usr/local/lib/python3.11/dist-packages (0.3.34)
Requirement already satisfied: langchain-core<0.4,>=0.1 in /usr/local/lib/python3.11/dist-packages (from langgraph) (0.3.55)
Requirement already satisfied: langgraph-checkpoint<3.0.0,>=2.0.10 in /usr/local/lib/python3.11/dist-packages (from langgraph) (2.0.25)
Requirement already satisfied: langgraph-prebuilt<0.2,>=0.1.8 in /usr/local/lib/python3.11/dist-packages (from langgraph) (0.1.8)
Requirement already satisfied: langgraph-sdk<0.2.0,>=0.1.42 in /usr/local/lib/python3.11/dist-packages (from langgraph) (0.1.63)
Requirement already satisfied: xxhash<4.0.0,>=3.5.0 in /usr/local/lib/python3.11/dist-packages (from langgraph) (3.5.0)
Requirement already satisfied: langsmith<0.4,>=0.1.125 in /usr/local/lib/python3.11/dist-packages (from langchain-core<0.4,>=0.1->langgraph) (0.1.125)
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in /usr/local/lib/python3.11/dist-packages (from langchain-core<0.4,>=0.1->langgraph) (8.5.0)
Requirement already satisfied: jsonpatch<2.0,>=1.33 in /usr/local/lib/python3.11/dist-packages (from langchain-core<0.4,>=0.1->langgraph) (1.33)
Requirement already satisfied: PyYAML>=5.3 in /usr/local/lib/python3.11/dist-packages (from langchain-core<0.4,>=0.1->langgraph) (6.0.2)
Requirement already satisfied: packaging<25,>=23.2 in /usr/local/lib/python3.11/dist-packages (from langchain-core<0.4,>=0.1->langgraph) (25.0)
Requirement already satisfied: typing-extensions>=4.7 in /usr/local/lib/python3.11/dist-packages (from langchain-core<0.4,>=0.1->langgraph) (4.12.2)
Requirement already satisfied: pydantic<3.0.0,>=2.5.2 in /usr/local/lib/python3.11/dist-packages (from langchain-core<0.4,>=0.1->langgraph) (2.10.6)
Requirement already satisfied: ormsgpack<2.0.0,>=1.8.0 in /usr/local/lib/python3.11/dist-packages (from langgraph-checkpoint<3.0.0,>=2.0.10->langgraph) (1.10.0)
Requirement already satisfied: httpx>=0.25.2 in /usr/local/lib/python3.11/dist-packages (from langgraph-sdk<0.2.0,>=0.1.42->langgraph) (0.27.0)
Requirement already satisfied: orjson>=3.10.1 in /usr/local/lib/python3.11/dist-packages (from langgraph-sdk<0.2.0,>=0.1.42->langgraph) (3.10.1)
Requirement already satisfied: anyio in /usr/local/lib/python3.11/dist-packages (from httpx>=0.25.2->langgraph-sdk<0.2.0,>=0.1.42->langgraph) (4.6.2)
Requirement already satisfied: certifi in /usr/local/lib/python3.11/dist-packages (from httpx>=0.25.2->langgraph-sdk<0.2.0,>=0.1.42->langgraph) (2025.1.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.11/dist-packages (from httpx>=0.25.2->langgraph-sdk<0.2.0,>=0.1.42->langgraph) (1.0.7)
Requirement already satisfied: idna in /usr/local/lib/python3.11/dist-packages (from httpx>=0.25.2->langgraph-sdk<0.2.0,>=0.1.42->langgraph) (3.10.1)
Requirement already satisfied: h11<0.15,>=0.13 in /usr/local/lib/python3.11/dist-packages (from httpcore==1.*->httpx>=0.25.2->langgraph-sdk<0.2.0,>=0.1.42->langgraph) (0.14.0)
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.11/dist-packages (from jsonpatch<2.0,>=1.33->langchain-core<0.4,>=0.1->langgraph) (3.0.0)
Requirement already satisfied: requests<3,>=2 in /usr/local/lib/python3.11/dist-packages (from langsmith<0.4,>=0.1.125->langchain-core) (2.32.3)
Requirement already satisfied: requests-toolbelt<2.0.0,>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from langsmith<0.4,>=0.1.125->langchain-core) (1.0.0)
Requirement already satisfied: zstandard<0.24.0,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from langsmith<0.4,>=0.1.125->langchain-core) (0.24.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.11/dist-packages (from pydantic<3.0.0,>=2.5.2->langchain-core) (0.7.0)
Requirement already satisfied: pydantic-core==2.33.1 in /usr/local/lib/python3.11/dist-packages (from pydantic<3.0.0,>=2.5.2->langchain-core) (2.33.1)
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.11/dist-packages (from pydantic<3.0.0,>=2.5.2->langchain-core) (0.4.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2->langsmith<0.4,>=0.1.125->langchain-core) (3.4.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2->langsmith<0.4,>=0.1.125->langchain-core) (2.3.0)
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.11/dist-packages (from anyio>httpx>=0.25.2->langgraph-sdk<0.2.0,>=0.1.42->langgraph) (1.3.1)
WARNING: Ignoring invalid distribution ~vidia-cublas-cu12 (/usr/local/lib/python3.11/dist-packages)
```

```

from langgraph.graph import MessagesState

class State(MessagesState):
    next: str

from typing import Annotated

itinerary_agent_prompt = """
You are an expert travel planner. Your task is to create a detailed itinerary for a trip based on the information provided by the user. The
- Day-by-day schedule with activities and locations
- Suggested times for each activity
- Recommendations for meals and restaurants
- Transportation options between locations
- Any special events or local attractions to consider
- Accommodation details if provided
Ensure the itinerary is well-organized, practical, and tailored to the user's preferences and constraints. Use a friendly and informative to
"""

def itinerary_agent(itinary_info: Annotated[str, "All the itinerary information in a plain text format"]):
    messages = [
        ("system", itinerary_agent_prompt),
        ("human", {itinary_info})
    ]

    return llm.invoke(messages)

```

```

pip install google-search-results

```

```

WARNING: Ignoring invalid distribution ~vidia-cublas-cu12 (/usr/local/lib/python3.11/dist-packages)
WARNING: Ignoring invalid distribution ~vidia-cublas-cu12 (/usr/local/lib/python3.11/dist-packages)
Requirement already satisfied: google-search-results in /usr/local/lib/python3.11/dist-packages (2.4.2)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from google-search-results) (2.32.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->google-search-results) (3.10)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->google-search-results) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->google-search-results) (2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->google-search-results) (202
WARNING: Ignoring invalid distribution ~vidia-cublas-cu12 (/usr/local/lib/python3.11/dist-packages)

```

```

#Flight_Agent#
from typing import Annotated, Literal, Optional
from typing_extensions import TypedDict
from serpapi import GoogleSearch
from langchain.prompts import ChatPromptTemplate
from langchain_core.tools import tool

flight_agent_prompt = """
You are an expert at extracting information out of plain text. The response should be in JSON format.
You need to extract the following parameters:
- departure_id - city/airport IATA code from which the traveler will depart, e.g. BOS for Boston
- arrival_id - city/airport IATA code from which the traveler is going, e.g. BOS for Boston
- outbound_date - the date on which the traveler will depart from the origin to go to the destination. Dates are specified in the ISO 8601 Y
- return_date - the date on which the traveller will return from the destination back to the origin. Date is specified in the ISO 8601 YYYY-
"""

class Flight_parameters(TypedDict):
    departure_id: Annotated[str, ..., "city/airport IATA code from which the traveler will depart, e.g. BOS for Boston"]
    arrival_id: Annotated[str, ..., "city/airport IATA code from which the traveler is going, e.g. BOS for Boston"]
    outbound_date: Annotated[str, ..., "the date on which the traveler will depart from the origin to go to the destination. Dates are speci
    return_date: Annotated[Optional[str], None, "the date on which the traveller will return from the destination back to the origin. Date i

flight_prompt = ChatPromptTemplate.from_messages([('system', flight_agent_prompt), ('human', '{input}')]
structured_llm_flight = flight_prompt | llm.with_structured_output(Flight_parameters)

@tool
def flight_tool(parameters: Annotated[str, "All the flight parameters in a plain text format"]):
    """
    Get the list of available flights based on users' requirements.
    Parameters:
        query (str): User's flight requirements as natural language input.
    Returns:
        str: A human-readable response with flight offers or an error message.
    """

```

```

"""
flight_parameters = structured_llm_flight.invoke({'input': parameters})
flight_parameters['engine'] = 'google_flights'
flight_parameters['currencyCode'] = 'USD'
flight_parameters['hl'] = 'en'
flight_parameters['api_key'] = ''

search = GoogleSearch(flight_parameters)
results = search.get_dict()
return json.dumps(results)

from langgraph.prebuilt import create_react_agent

flight_agent = create_react_agent(
    llm,
    tools=[flight_tool],
    prompt="""
        You are a flight agent. You are responsible for finding the best flights for the user. Analyze the user's requirements and the tool
    """
)

#Hotel_Agent#
from typing import Annotated, Literal, Optional
from typing_extensions import TypedDict
from serpapi import GoogleSearch
from langchain.prompts import ChatPromptTemplate
from langchain_core.tools import tool
import json

hotel_agent_prompt = """
You are an expert at extracting information out of plain text. The response should be in JSON format.
You need to extract the following parameters:
- city - city name where the hotel is located, e.g. New York,Tokyo,Paris
- check_in_date - the check-in date in ISO 8601 YYYY-MM-DD format
- check_out_date - the check-out date in ISO 8601 YYYY-MM-DD format
- num_adults - number of adults staying
- num_rooms - number of rooms needed
"""

class HotelParameters(TypedDict):
    city: Annotated[str, ..., "City name where the hotel is located, e.g. NYC for New York,Tokyo,Paris"]
    check_in_date: Annotated[str, ..., "Check-in date in ISO 8601 YYYY-MM-DD format"]
    check_out_date: Annotated[str, ..., "Check-out date in ISO 8601 YYYY-MM-DD format"]
    num_adults: Annotated[int, ..., "Number of adults"]
    num_rooms: Annotated[int, ..., "Number of rooms"]

hotel_prompt = ChatPromptTemplate.from_messages([
    ('system', hotel_agent_prompt),
    ('human', '{input}')
])
structured_llm_hotel = hotel_prompt | llm.with_structured_output(HotelParameters)

@tool
def hotel_tool(parameters: Annotated[str, "All the hotel booking parameters in a plain text format"]):
    """
    Get the list of available hotels based on users' requirements.
    Parameters:
        parameters (str): User's hotel requirements as natural language input.
    Returns:
        str: A human-readable response with hotel offers or an error message.
    """
    hotel_parameters = structured_llm_hotel.invoke({'input': parameters})
    hotel_parameters['engine'] = 'google_hotels' # Hypothetical engine, depends on SerpAPI support
    hotel_parameters['currencyCode'] = 'USD'
    hotel_parameters['hl'] = 'en'
    hotel_parameters['api_key'] = ''

    search = GoogleSearch(hotel_parameters)
    results = search.get_dict()
    return json.dumps(results)

from langgraph.prebuilt import create_react_agent

hotel_agent = create_react_agent(

```

```

    llm,
    tools=[hotel_tool],
    prompt="""
        You are a hotel agent. You are responsible for finding the best hotel options for the user.
        Analyze the user's requirements and the tool response and return a few of the best hotel offers.
    """
)

from langgraph.graph import END
from langgraph.types import Command
from typing_extensions import TypedDict
from typing import Literal

chatbot_prompt = """
You are a dedicated travel planning chatbot designed to create personalized and well-organized trip itineraries based on user preferences and
Your Responsibilities:
1. Greeting & Introduction: Start by greeting the user and explaining that you're here to help plan their trip. Inform them what task you can help with.
    Currently you can help them with 1. flight information 2. itinerary planning 3. Hotel information
2. Gather Preferences & Constraints: Ask the user about their travel preferences, such as destinations, budget, duration, interests (e.g., a
3. Information Collection: Inquire about specific details that is required for each agent as follows -
    - hotel_agent: City, check-in date, check-out date, number of adults, and number of rooms.
    - flight_agent: Departure and arrival city/airport IATA code, departure and return date.
    - itinerary_agent: preferred activities, types of accommodations, transportation preferences, and meal preferences.
    If any information is missing, ask the user for the missing information. Only move on to routing once the user has provided all the info
4. Agent Routing: Based on the collected information, determine which agent to route the user to:
    - itinerary_agent: If sufficient trip-related information is provided or if the user asks you to come up with the rest of the information.
    - flight_agent: If the user asks for flight information.
    - hotel_agent: If the user asks for hotel information.
    - human_interrupt: This is the user itself and not an expert agent. Only give response that are crafted by you or any request for additional
5. Response Handling:
    - Structured Output: Ensure all responses are in JSON format with the keys:
        - `next`: The next agent to route to (`itinerary_agent`, `human_interrupt`, or `FINISH`).
        - `messages`: Any messages to send to the user.
    - Information Validation: If any required information is missing or incomplete, gather more information from the user.
6. Finalization: Once the itinerary is complete, ask the user if they would like to add anything else. If not, respond with `FINISH` in the
Communication Style:
- Use a friendly, clear, and informative tone.
- Ensure all questions are concise and easy to understand.
- Make the user feel supported and assured throughout the planning process.
- When you are done with the itinerary, always route to human_interrupt agent.
Example Workflow:
1. Chatbot: "Hello! I'm here to help you plan your perfect trip. Could you please share your preferred destinations and travel dates?"
2. User: [Provides information]
3. Chatbot: [Routes to `itinerary_agent` with collected information]
4. itinerary_agent: [Creates itinerary]
5. Chatbot: "Here's your personalized itinerary! Here is the itinerary. Would you like to add anything else?"
6. User: "No, that's all. Thank you!"
7. Chatbot: `FINISH`
Ensure seamless and natural interactions while effectively gathering all necessary information to create a comprehensive trip itinerary.
"""

class Router(TypedDict):
    """Worker to route to next. If no workers needed, route to FINISH."""

    next: Literal['itinerary_agent', 'human_interrupt', 'FINISH']
    messages: str

def chatbot_node(state: State) -> Command[Literal['itinerary_agent', 'human_interrupt', 'flight_agent', 'hotel_agent', "__end__"]]:
    messages = [
        {"role": "system", "content": chatbot_prompt},
    ] + state["messages"]

    response = llm.with_structured_output(Router).invoke(messages)

    goto = response["next"]
    if goto == "FINISH":
        goto = END

    return Command(goto=goto, update={"next": goto, "messages": response["messages"]})

from langgraph.types import Command
from langchain_core.prompts.chat import HumanMessage
from typing_extensions import Literal

def itinerary_node(state: State) -> Command[Literal['chatbot']]:
    # Convert the state messages to a list of strings

```

```

human_messages = [msg.content for msg in state['messages'] if isinstance(msg, HumanMessage)]

messages = [
    ("system", itinerary_agent_prompt),
    ("human", " ".join(human_messages)) # Join all human messages into a single string
]

response = llm.invoke(messages)

return Command(goto='chatbot', update={"messages": [response.content]})

def flight_node(state: State) -> Command[Literal['chatbot']]:
    result = flight_agent.invoke(state)
    return Command(
        update={
            "messages": [result["messages"][-1].content]
        },
        goto="chatbot",
    )

def hotel_node(state: State) -> Command[Literal['chatbot']]:
    result = hotel_agent.invoke(state)
    return Command(
        update={
            "messages": [result["messages"][-1].content]
        },
        goto="chatbot",
    )

from langgraph.types import Command
from typing import Literal

def human_interrupt(state: State) -> Command[Literal['chatbot']]:

    query = state['messages'][-1].content

    user_input = input("user: ")

    return Command(goto='chatbot', update={"messages": [user_input]})

from langgraph.graph import StateGraph, START

if __name__ == "__main__":
    builder = StateGraph(State)
    builder.add_edge(START, "chatbot")
    builder.add_node("chatbot", chatbot_node)
    builder.add_node("itinerary_agent", itinerary_node)
    builder.add_node("flight_agent", flight_node)
    builder.add_node("hotel_agent", hotel_node)
    builder.add_node("human_interrupt", human_interrupt)

    graph = builder.compile()

    for s in graph.stream({"messages": [("user", "Hi")]}, subgraphs=True):
        for key, value in s[1].items():
            if key in ['chatbot', 'itinerary_agent', 'flight_agent', 'hotel_agent']:
                print(value['messages'])
        print("----")

```